P R O J E C T R E P O R T

Design and Implementation of a

8-bit RISC Processor with a 5-Stage Pipeline:

Student	Bùi Hữu Lợi
Student ID	SE196720
University	FPT University
Major	IC Design
Tools	Xilinx Vivado 2026.1 (XSim), Verilog HDL
Target device	Artix-7 xc7a35tcpg236-1
Date	October 2026

A Harvard-architecture 8-bit RISC core with 16 instructions and a classic IF / ID / EX / MEM / WB pipeline passes its self-checking testbench in Vivado 2026.1 behavioral simulation and completes synthesis. Every hazard is resolved in hardware: forwarding, a register-file bypass, a 1-cycle load-use stall and a 2-cycle branch flush.

1. Specification

Parameter	Value
Data width / registers	8 bit; R0–R7 (R0 is an ordinary register)
Instruction width	16 bit, fixed; 4-bit opcode, 16 instructions
Memories (Harvard)	Instruction ROM 256 × 16 bit; data RAM 256 × 8 bit
Program counter	8 bit (256 instructions)
Flags	Z (zero), C (carry / borrow / shifted-out bit)
Pipeline	5 stages, up to 5 instructions in flight
Hazard handling	Forwarding, bypass, 1-cycle load-use stall, 2-cycle branch flush
Target	Xilinx FPGA via Vivado; synthesizable Verilog-2001

2. Instruction set

Only LD and ST touch memory (load/store RISC). The opcode and the register fields sit at the same bit positions in every format, so the decoder can read them before it knows which instruction it has.

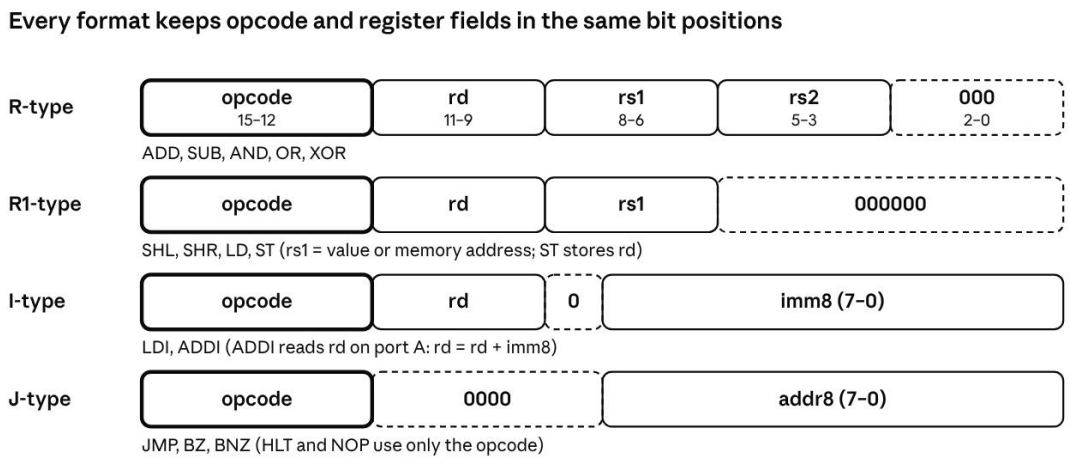


Figure 1 · instruction formats, bit 15 on the left

Figure 1. The four 16-bit formats; the heavy outline marks the opcode in every row and dashed boxes are fixed zero fields.

Class	Instructions	Operation	Flags
Arithmetic	ADD, SUB rd, rs1, rs2	rd = rs1 ± rs2	Z, C (borrow for SUB)
Logic	AND, OR, XOR rd, rs1, rs2	bitwise	Z
Shift	SHL, SHR rd, rs1	shift by 1; C = bit shifted out	Z, C
Immediate	LDI rd, imm8 / ADDI rd, imm8	rd = imm8 / rd = rd + imm8	ADDI: Z, C
Memory	LD rd, [rs1] / ST rd, [rs1]	rd = mem[rs1] / mem[rs1] = rd	none
Control	JMP / BZ / BNZ addr8	PC = addr8 (BZ if Z = 1, BNZ if Z = 0)	none
Misc	NOP, HLT	do nothing / stop	none

Opcodes: 0 NOP, 1 ADD, 2 SUB, 3 AND, 4 OR, 5 XOR, 6 SHL, 7 SHR, 8 LDI, 9 ADDI, A LD, B ST, C JMP, D BZ, E BNZ, F HLT. Example: `ADD R2, R2, R1` = 0001 010 010 001 000 = **0x1488**; `LDI R1, 0x3C` = **0x823C**.

3. System architecture

The core has two inputs (clk, rst) and four observation outputs (halted, pc_out, flags_out, instr_done). The program is built into imem.v, so nothing external is needed.

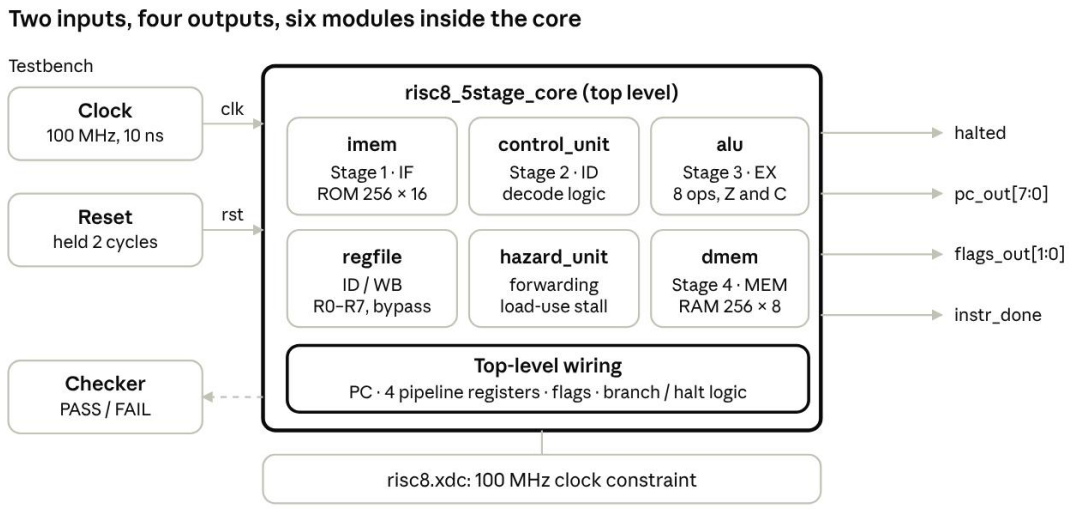


Figure 2 · system block diagram

Figure 2. The testbench drives clk and rst and checks the outputs; the core instantiates six modules; the .xdc file sets the clock for Vivado.

4. Pipeline and datapath

IF reads the instruction at PC; ID decodes and reads registers; EX runs the ALU, updates flags and decides branches; MEM performs LD/ST; WB writes the register. Putting the ALU and data memory in separate stages balances stage delay. Every pipeline register carries a **valid** bit: valid = 0 is a bubble whose write-enables are forced to 0, so it can never change a register, memory or flag.

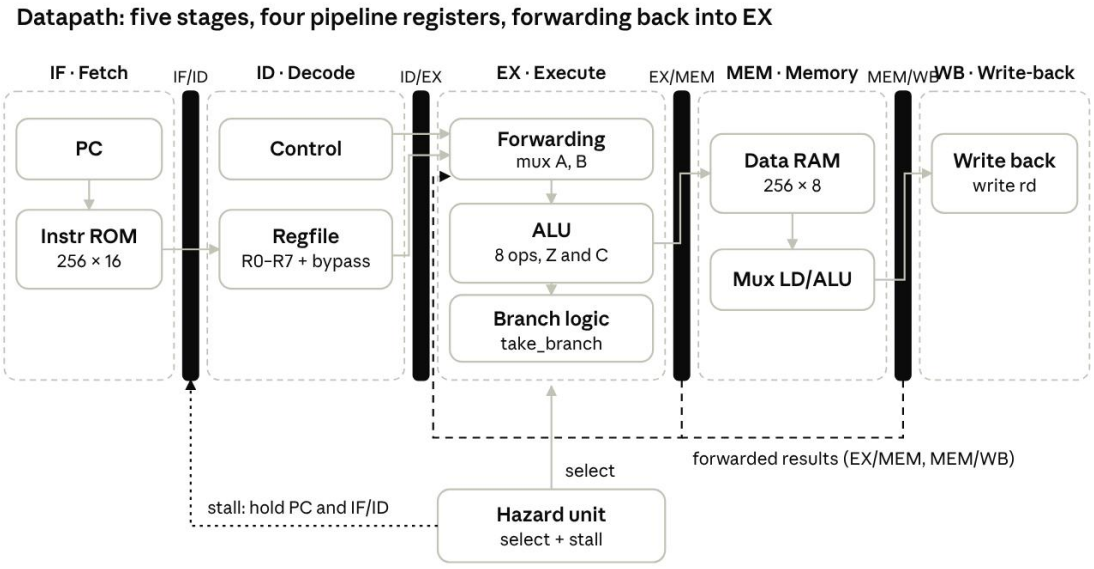


Figure 3 · five-stage datapath

Figure 3. Complete datapath. Solid bars are the four pipeline registers, the dashed path carries forwarded results from EX/MEM and MEM/WB back into EX, and the dotted line is the stall that holds PC and IF/ID. A taken branch in EX reloads the PC and flushes IF/ID and ID/EX; WB data returns to the register file.

Pipeline register	Contents	Width
IF/ID	instruction, valid	17 bit
ID/EX	valid, 14 control bits, rd, two source numbers, two operand values, imm8	48 bit
EX/MEM	valid, reg_we, mem_to_reg, mem_we, halt, rd, ALU result, address, store data	32 bit
MEM/WB	valid, reg_we, halt, rd, result	14 bit

Next-PC selection uses a fixed priority:

Priority	Condition	Next PC	IF/ID receives
1	Branch or jump taken in EX	target	bubble
2	HLT reached EX	hold	bubble
3	Load-use stall	hold	hold
4	Normal	PC + 1	fetched instruction

5. Control unit

The control unit is a pure combinational lookup from opcode to signals; every signal defaults to 0 (a NOP) and each instruction turns on only what it needs. Decoding happens in ID, but the signals are consumed later, so they ride along in the pipeline registers.

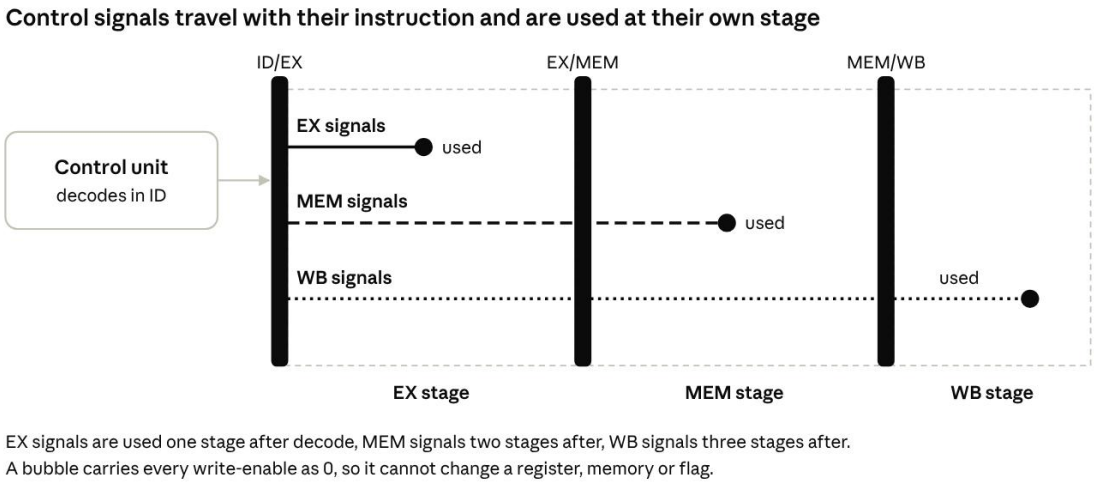


Figure 4 · control signals in the pipeline

Figure 4. Signals are decoded once in ID and copied through ID/EX, EX/MEM and MEM/WB; each stage uses only its own group.

Group	Signals	Used in
EX	alu_op (3 bit), alu_src_imm, flag_we, jump, br_z, br_nz, halt	Stage 3
MEM	mem_we, mem_to_reg	Stage 4
WB	reg_we, rd	Stage 5
ID only	ra_is_rd, rb_is_rd (ADDI reads rd on port A; ST reads rd on port B), uses_a, uses_b (gate forwarding and stalls)	Stage 2

6. Hazards

Kind	Cause	Solution	Cost
Structural	Two stages need one resource	Harvard memories; separate register read/write ports	0
Data (RAW)	Result not yet written back	Forwarding from EX/MEM and MEM/WB; register-file bypass	0 cycles
Data (load-use)	Loaded value needed by the next instruction	1-cycle stall, then forwarding	1 cycle
Control	Taken branch makes fetched instructions wrong	Flush the two younger instructions	2 cycles

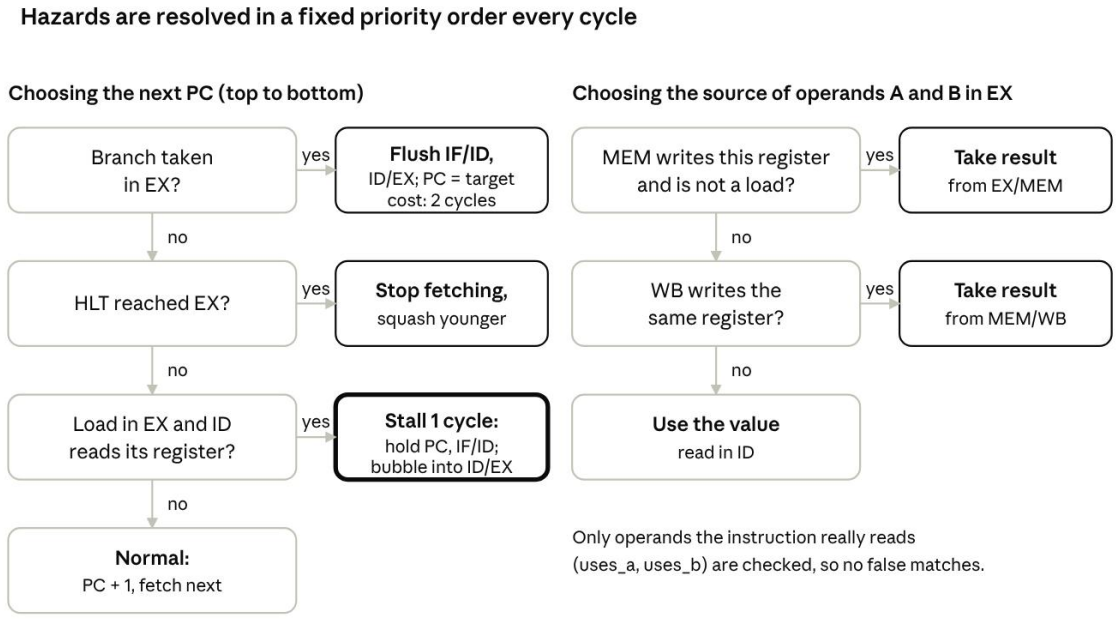


Figure 5 · hazard decision flowchart

Figure 5. Left: next-PC selection, where a taken branch outranks halt and halt outranks a load-use stall. Right: forwarding rules, applied separately to operand A and operand B.

6.1 Forwarding and bypass

For each operand the hazard unit picks the newest value: EX/MEM if MEM writes the same register and is not a load, else MEM/WB if WB writes it, else the value read in ID. Only operands the instruction really reads (uses_a, uses_b) are checked, so unused fields never match falsely. The register file passes a value being written straight to a reader in the same cycle, which covers a producer three instructions earlier.

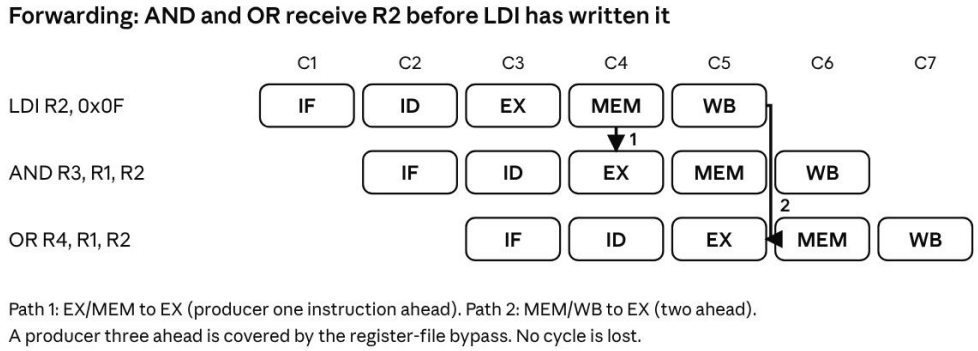


Figure 6 · forwarding timing

Figure 6. Forwarding timeline. Each cell is labelled with its stage; the arrows show R2 skipping the register file.

6.2 Load-use stall

A load has its data only at the end of MEM. If the next instruction needs it in EX, the hazard unit stalls one cycle: PC and IF/ID hold and a bubble enters ID/EX.

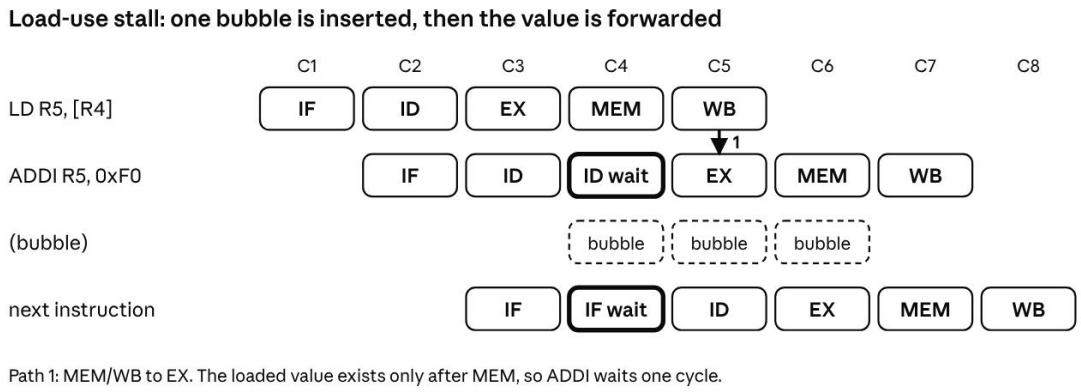


Figure 7 · load-use stall timing

Figure 7. Load-use stall. Thick-outlined cells are the held stages; the dashed cells are the bubble that travels through EX, MEM and WB without effect.

6.3 Branch flush and halt

Branches resolve in EX, where the flags are current. A taken branch loads the PC with the target and turns the two younger instructions into bubbles, so they never reach MEM or WB; not-taken branches cost nothing. When HLT reaches EX, fetching stops and younger instructions are squashed while older ones finish; when HLT reaches WB, **halted** goes to 1 and the pipeline freezes.

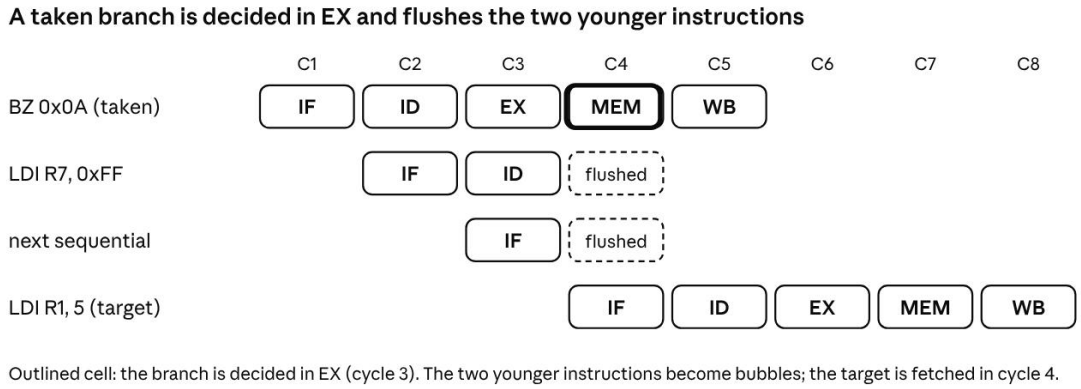


Figure 8 · taken-branch flush timing

Figure 8. Taken-branch flush: two wrong-path instructions are cancelled before MEM, so a taken branch costs 2 cycles.

7. Instruction walk-through

Stage	ADD R3, R1, R2	LD R5, [R4]	ST R2, [R4]	BZ 0x0A
IF	fetch 0x1650	fetch 0xAB00	fetch 0xB500	fetch 0xD00A
ID	read R1, R2; reg_we, flag_we	read R4; reg_we, mem_to_reg	read R4 (A), R2 (B); mem_we	br_z = 1
EX	R1 + R2, set Z, C	address = R4	address = R4, data = R2	if Z = 1: PC = 0x0A, flush 2
MEM	pass result	read mem[R4]	write mem[R4] = R2	none
WB	R3 = result	R5 = loaded byte	none	none

8. Key design decisions

Decision	Why	Alternative
Branch resolved in EX	Z comes from the ALU, so the decision always uses current flags	Resolve in ID: 1-cycle penalty, extra compare hardware
Asynchronous memory read	Data available in the same cycle as the address	Synchronous block RAM: one more cycle of latency
Write-through register	Removes the distance-3 hazard	Write in first half, read in second half

Decision	Why	Alternative
bypass	with one small mux	of the cycle
Forwarding gated by uses_a / uses_b	No false forwards or stalls	Simpler logic, occasional needless stalls
Program inside imem.v	No external file to lose	$readmemh from a .hex file

9. Simulation results (Vivado 2026.1)

The core was simulated in Vivado 2026.1 (project 8BitRiscCPU) with the self-checking testbench tb_risc8_5stage.v. The testbench resets the core, prints one line per clock (time, PC, halted, flags), waits for the HLT instruction, then verifies the register file and data memory and prints PASS or FAIL. All numbers below are read from the Tcl console and waveform of that run.

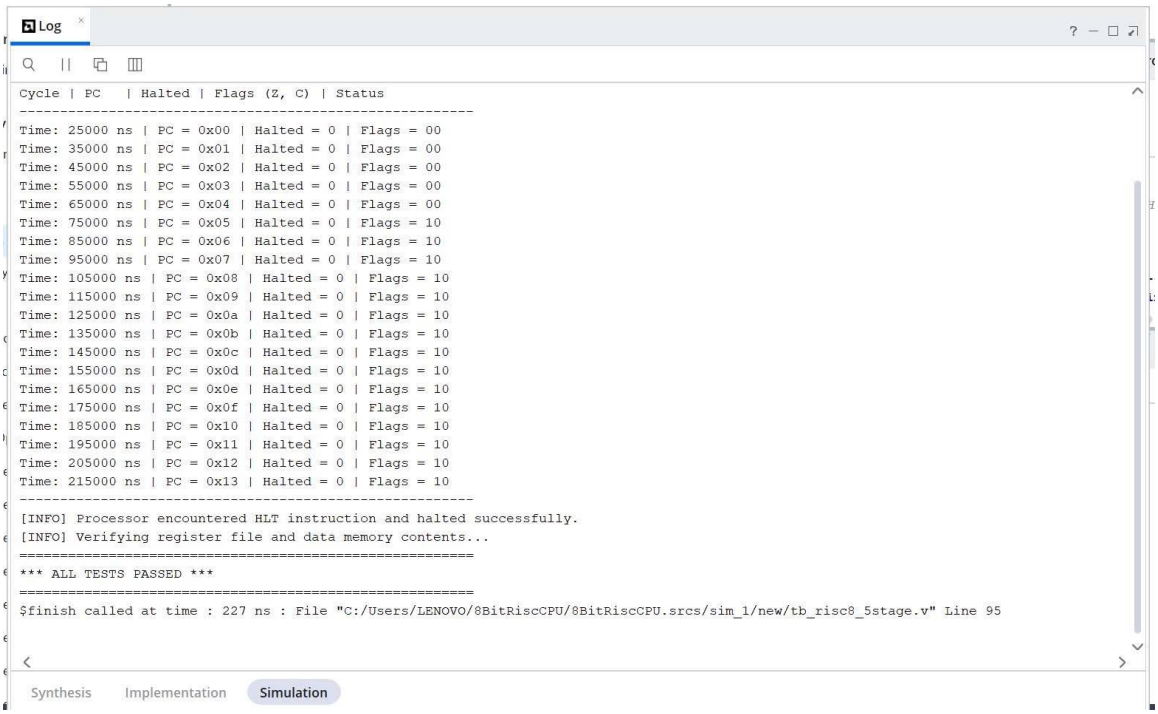


Figure 9. Console log of tb_risc8_5stage. PC advances one address per clock from 0x00 to 0x13, the flags change from 00 to 10 when PC reaches 0x05, the HLT instruction is detected, registers and data memory are verified, and the testbench prints ALL TESTS PASSED. $finish is called at 227 ns.

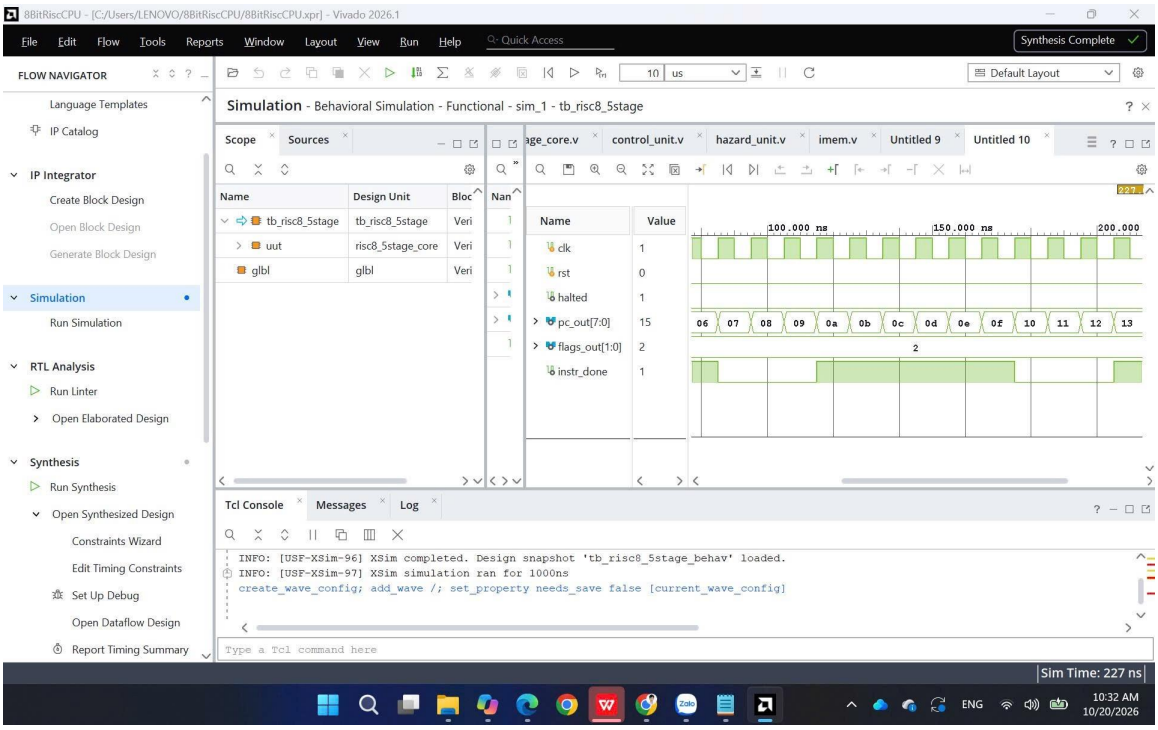


Figure 10. Waveform of the same run. At the end halted = 1 and rst = 0, pc_out counts 0x06 to 0x13, flags_out = 2 (binary 10) and instr_done is high while instructions complete.

Observation in this run	Value
Simulator / top	Vivado 2026.1 behavioral simulation; tb_risc8_5stage instantiates uut = risc8_5stage_core
PC range	0x00 to 0x13, one address per 10 ns clock (no taken branch visible in the trace)
Flags (Z, C)	00 until PC = 0x05, then 10 (Z = 1, C = 0) for the rest of the run
End of run	HLT detected, halted = 1, ALL TESTS PASSED, $finish at 227 ns

What the run shows. Throughput: PC steps by exactly one address on every 10 ns clock, with no jump and no repeated value, so in the logged window the pipeline never flushed or stalled and fetched one instruction per cycle (ideal CPI of 1). Flags: flags_out stays 00 while the first instructions fill the pipeline and becomes 10 once the first flag-setting instruction has passed EX, about three fetches after it was fetched, because flags are written at the end of EX; no later instruction changes them. Halt: when HLT reaches EX fetching stops, and when it reaches WB halted rises and the pipeline freezes; the testbench then reports that the processor halted, verifies the register file and data memory and prints ALL TESTS PASSED, and $finish is called at 227 ns. The log does not print an instruction count, so an exact CPI is not derived here.

10. Synthesis and implementation in Vivado

The design goes through the standard Vivado steps: behavioral simulation, synthesis (RTL to LUTs and flip-flops, no latch warnings allowed), implementation (place and route; worst negative slack must be at

least 0 at 100 MHz) and bitstream generation (needs pin assignments for clk, rst and the LED outputs). Synthesis completed in the run below (status bar: Synthesis Complete).

↗ Figure 11 · Vivado flow

Figure 11. Vivado flow. Simulation is highlighted because synthesis and implementation are only worth running once the testbench reports PASS.

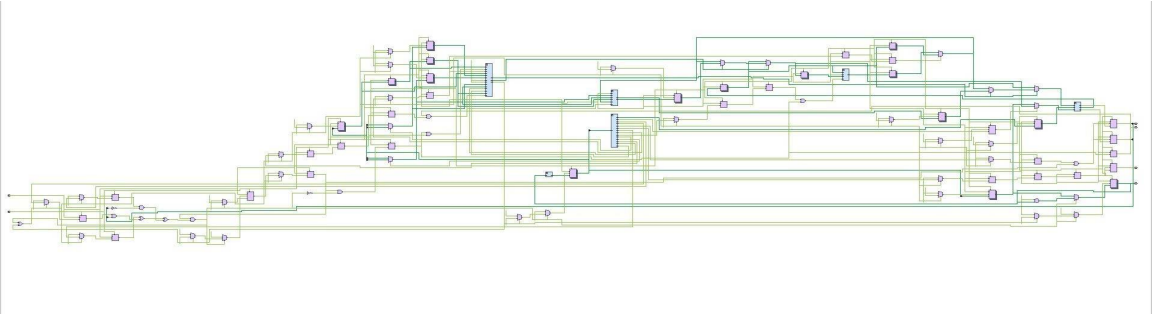


Figure 12. RTL (elaborated) schematic: the Verilog drawn as generic gates, multiplexers and registers before technology mapping, with clk and rst entering on the left and the outputs on the right.

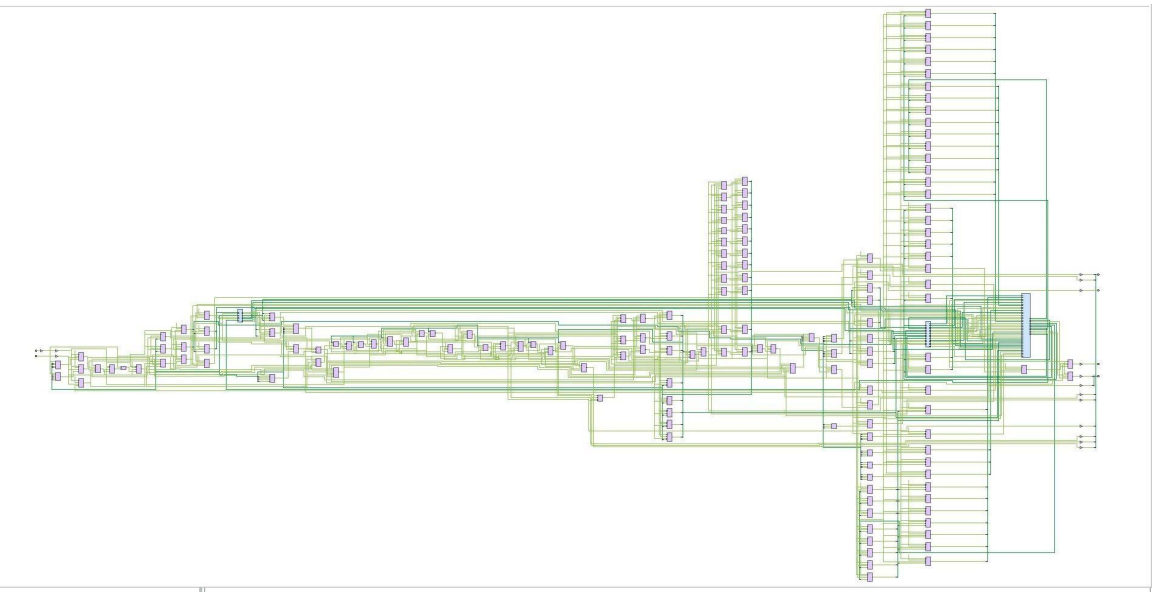


Figure 13. Synthesized schematic: after synthesis the same design is mapped to FPGA primitives, giving a much larger netlist; the wide columns of identical cells are typical of register-file and data-memory bits.

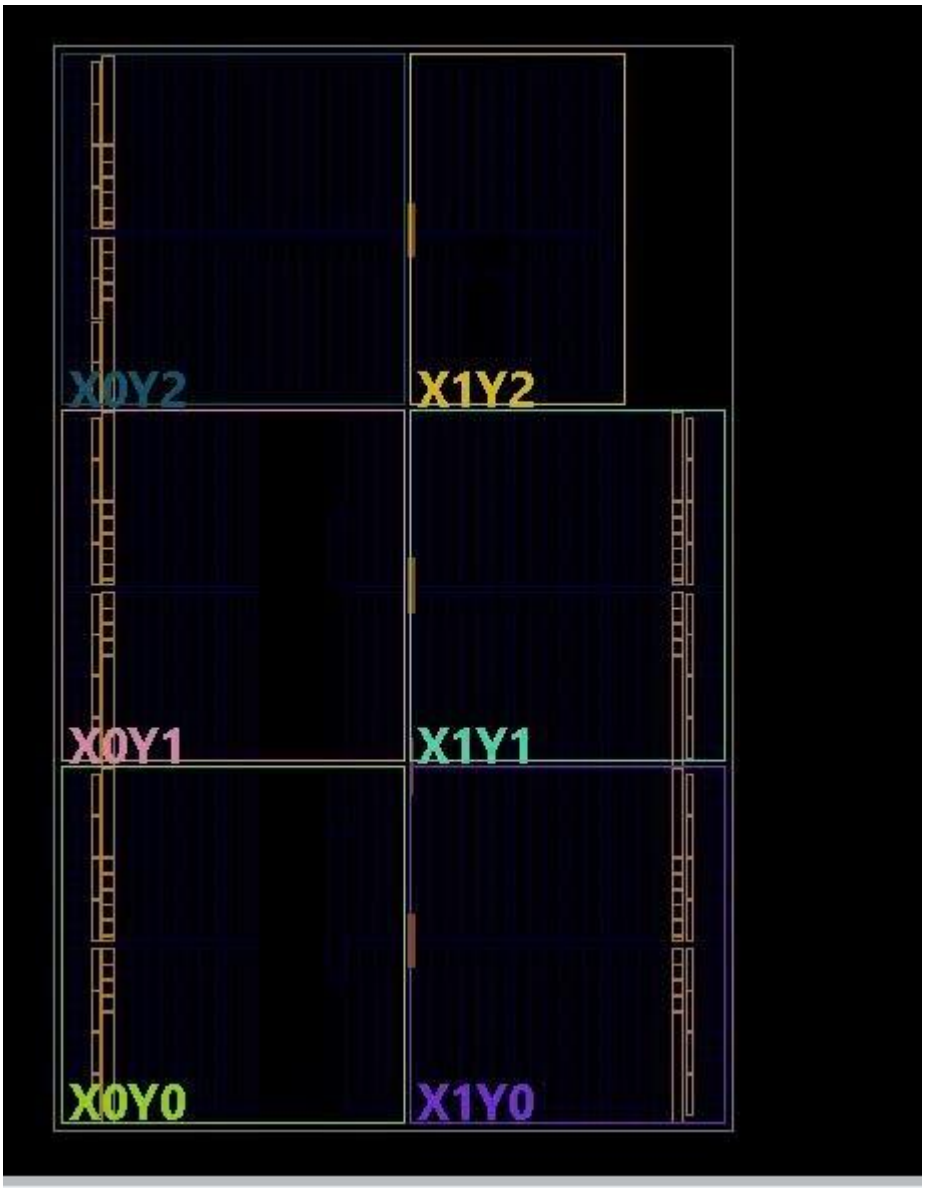


Figure 14. Device view of the target FPGA, showing its six clock regions (X0Y0 to X1Y2) where implementation places the design.

Timing summary and utilization reports are not among the supplied screenshots, so no clock-frequency or resource numbers are claimed here. Run Report Timing Summary (WNS at least 0) and Report Utilization after implementation to add them.

11. Limitations and next steps

- No branch prediction: every taken branch costs 2 flush cycles. No taken branch appears in the logged run, so that penalty is not exercised there.
- 8-bit PC and addresses limit the machine to 256 instructions and 256 data bytes; the program is hard-coded in imem.v.
- Only Z and C flags, no interrupts. Run Report Timing Summary in Vivado to confirm WNS is at least 0 at 100 MHz on the target board.

- Next: static branch prediction, $readmemh program loading, CALL/RET, N and V flags, a clock divider for LED demos.
- Verification gap: count instr_done pulses and clock cycles in the testbench to print CPI, and add a test program with load-use pairs and taken branches to exercise the stall and flush paths.